

VÕ TIỀN

Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



Kỹ Thuật Lập Trình (Cơ bản và nâng cao C++)

KTILT2 - HK242

TASK 1 BTL2: 3.1 - 3.2 - 3.3 - 3.6

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>

Mục lục

1	Lý thuyết	2
2	Bài Tập Lý thuyết	2
3	Câu hỏi thiết kế BTL cơ bản (CS, CE, DS)	6
4	Câu hỏi thiết kế BTL trung bình (CS, CE)	6
5	Câu hỏi thiết kế mới (Mức nâng cao - Tính mở rộng và bảo trì) (CS)	7
6	Design Patterns (mẫu thiết kế) và nguyên tắc SOLID (CS)	7



1 Lý thuyết

- TASK 7 Nền móng OOP - Xây dựng thế giới đối tượng
- TASK 8 Kiến trúc sư OOP - Kiểm soát và tổ chức
- TASK 9 Nghệ thuật lập trình - Đa hình và Trừu tượng hóa

2 Bài Tập Lý thuyết

1. Trong C++, từ khóa `virtual` được sử dụng để:
 - a) Tạo một lớp trừu tượng
 - b) Cho phép ghi đè phương thức trong lớp con
 - c) Ngăn chặn phương thức bị ghi đè
 - d) Tạo đối tượng của lớp cha
2. Khi nào một lớp cần có **hàm hủy ảo**?
 - a) Khi nó có constructor
 - b) Khi nó có con trỏ trở về tài nguyên cấp phát động
 - c) Khi nó có phương thức thuần ảo
 - d) Khi nó có nhiều phương thức `static`
3. Lợi ích của việc sử dụng con trỏ của lớp cha để trỏ đến đối tượng lớp con là gì?
 - a) Cho phép truy cập tất cả phương thức của lớp con
 - b) Cho phép sử dụng tính đa hình (polymorphism)
 - c) Giúp tăng tốc độ thực thi chương trình
 - d) Giảm kích thước bộ nhớ của đối tượng
4. Khi một lớp kế thừa từ một lớp khác nhưng không ghi đè phương thức ảo, điều gì xảy ra?
 - a) Chương trình sẽ báo lỗi biên dịch
 - b) Phương thức của lớp cha sẽ được gọi khi sử dụng con trỏ của lớp cha
 - c) Phương thức của lớp con vẫn được gọi bình thường
 - d) Không có phương thức nào được gọi
5. Từ khóa `override` trong C++ có tác dụng gì?
 - a) Bắt buộc phương thức phải ghi đè
 - b) Ngăn chặn ghi đè phương thức
 - c) Cho phép một phương thức gọi phương thức cùng tên của lớp cha
 - d) Không có tác dụng đặc biệt
6. Kết quả của đoạn code sau là gì?

```
1 class A {
2 public:
3     virtual void show() { cout << "A"; }
4 };
5
6 class B : public A {
7 public:
8     void show() override { cout << "B"; }
9 };
10
11 int main() {
12     A* obj = new B();
13     obj->show();
14     delete obj;
15     return 0;
16 }
```



- a) In ra "A"
- b) In ra "B"
- c) Lỗi biên dịch
- d) Kết quả không xác định

7. Lớp BaseStructure đóng vai trò gì trong thiết kế sau?

```
1 class BaseStructure {
2 public:
3     virtual void insert(int value) = 0;
4 };
5
6 class LinkedList : public BaseStructure {
7 public:
8     void insert(int value) override {
9         cout << "Inserting " << value << " into LinkedList" << endl;
10    }
11};
```

- a) Interface cho các cấu trúc dữ liệu khác nhau
- b) Có thể tạo đối tượng trực tiếp
- c) Không thể ghi đè phương thức insert
- d) Thiếu từ khóa virtual ở lớp con

8. Kết quả của đoạn code sau là gì?

```
1 class BaseStructure {
2 public:
3     virtual void insert(int value) {
4         cout << "Base insert" << endl;
5     }
6 };
7
8 class Stack : public BaseStructure {
9 public:
10    void insert(int value) override {
11        cout << "Pushing " << value << " into Stack" << endl;
12    }
13};
14
15 int main() {
16     BaseStructure* ds = new Stack();
17     ds->insert(10);
18 }
```

- a) In ra "Base insert"
- b) In ra "Pushing 10 into Stack"
- c) Lỗi biên dịch
- d) Kết quả không xác định

9. Tại sao phương thức insert của Graph được gọi trong đoạn code sau?

```
1 class BaseStructure {
2 public:
3     virtual void insert(int value) = 0;
4 };
5
6 class Graph : public BaseStructure {
7 public:
8     void insert(int value) override {
```



```
9         cout << "Adding vertex " << value << " into Graph" << endl;
10     }
11 };
12
13 int main() {
14     BaseStructure* ds = new Graph();
15     ds->insert(5);
16 }
```

- a) Vì BaseStructure là interface b) Vì BaseStructure có phương thức ảo
c) Vì Graph ghi đè insert d) Cả B và C

10. Kết quả của đoạn code sau là gì?

```
1 class BaseStructure {
2 public:
3     virtual ~BaseStructure() {
4         cout << "BaseStructure destroyed" << endl;
5     }
6 };
7
8 class Tree : public BaseStructure {
9 public:
10     ~Tree() {
11         cout << "Tree destroyed" << endl;
12     }
13 };
14
15 int main() {
16     BaseStructure* ds = new Tree();
17     delete ds;
18 }
```

- a) Chỉ "BaseStructure destroyed" b) Chỉ "Tree destroyed"
c) "Tree destroyed" rồi "BaseStructure destroyed" d) Không có gì xảy ra

11. Tại sao `dynamic_cast<Queue*>(ds);` hợp lệ?

```
1 class BaseStructure {
2 public:
3     virtual void insert(int value) = 0;
4 };
5
6 class Queue : public BaseStructure {
7 public:
8     void insert(int value) override {
9         cout << "Enqueue " << value << endl;
10     }
11 };
12
13 int main() {
14     BaseStructure* ds = new Queue();
```



```
15 Queue* q = dynamic_cast<Queue*>(ds);  
16 q->insert(10);  
17 }
```

- a) Vì ds trỏ đến một đối tượng Queue b) Vì BaseStructure có phương thức ảo
c) Vì dynamic_cast chỉ dùng cho lớp có de- d) Cả A và B
structor ảo

12. Điều gì xảy ra nếu chạy đoạn code sau?

```
1 class BaseStructure {  
2 public:  
3     void show() {  
4         cout << "BaseStructure" << endl;  
5     }  
6 };  
7  
8 class HashTable : public BaseStructure {  
9 public:  
10    void show() {  
11        cout << "HashTable" << endl;  
12    }  
13 };  
14  
15 int main() {  
16     BaseStructure* ds = new HashTable();  
17     ds->show();  
18 }
```

- a) "BaseStructure" b) "HashTable"
c) Lỗi biên dịch d) Kết quả không xác định

13. Điều gì xảy ra khi khởi tạo Queue?

```
1 class Queue : public BaseStructure {  
2 public:  
3     Queue() {  
4         cout << "Queue constructor" << endl;  
5     }  
6 };
```

- a) Chỉ "Queue constructor" được in ra b) Lỗi vì BaseStructure không có constructor mặc định
c) "BaseStructure constructor" chạy trước "Queue constructor" d) Không có gì xảy ra

14. Điều gì xảy ra nếu ta viết Stack s;?

```
1 class BaseStructure {  
2 public:  
3     virtual void insert(int value) = 0;  
4 };
```



```
5  
6 class Stack : public BaseStructure {};
```

- a) Biên dịch thành công
b) Lỗi vì `Stack` không ghi đè `insert()`
c) Chương trình vẫn chạy nhưng `insert()` bị bỏ qua
d) `Stack` tự động kế thừa `insert()` từ `BaseStructure`

15. Câu nào đúng về phương thức `clear` trong đoạn code sau?

```
1 class BaseStructure {  
2 public:  
3     virtual void insert(int value) = 0;  
4     virtual void clear() { cout << "Base clear" << endl; }  
5 };
```

- a) Lỗi vì interface không thể có hàm có thân
b) `clear()` có thể bị ghi đè bởi lớp con
c) `clear()` chỉ có thể gọi từ `BaseStructure`
d) `clear()` chỉ có thể được định nghĩa bên ngoài lớp

3 Cấu hỏi thiết kế BTL cơ bản (CS, CE, DS)

- Tại sao lớp `Unit` được thiết kế như một lớp trừu tượng (abstract class) và điều này ảnh hưởng như thế nào đến các lớp `Vehicle` và `Infantry`?
- Trong phương thức `getAttackScore()` của lớp `Infantry`, việc cập nhật `quantity` dựa trên số cá nhân (digital root) có thể dẫn đến vấn đề gì nếu phương thức này được gọi nhiều lần, và làm thế nào để khắc phục?
- Việc sử dụng enum cho `VehicleType` và `InfantryType` thay vì chuỗi hoặc số nguyên thông thường mang lại lợi ích gì trong thiết kế này?
- Tại sao phương thức `str()` trong cả hai lớp `Vehicle` và `Infantry` sử dụng switch-case để chuyển đổi kiểu enum thành chuỗi, và có cách nào tối ưu hơn không?
- Nếu cần thêm một thuộc tính mới (ví dụ: `health` - sức khỏe) vào tất cả các đơn vị quân sự, bạn sẽ thêm nó vào lớp nào và tại sao? Cách triển khai cần thay đổi như thế nào?

4 Cấu hỏi thiết kế BTL trung bình (CS, CE)

- Làm thế nào để thiết kế lại lớp `Position` để hỗ trợ thêm các tọa độ 3D (ví dụ: thêm chiều cao) mà không phá vỡ các lớp hiện có như `Unit`, `Vehicle`, và `Infantry`?
- Nếu muốn thêm một cơ chế di chuyển cho các đơn vị quân sự, bạn sẽ thêm phương thức `move()` vào đâu và cách triển khai cơ bản sẽ như thế nào?
- Làm thế nào để tích hợp một hệ thống "trạng thái" (ví dụ: `READY`, `DAMAGED`, `DESTROYED`) cho các đơn vị quân sự, và điều này ảnh hưởng thế nào đến phương thức `getAttackScore()`?
- Nếu muốn lưu trữ lịch sử vị trí của mỗi đơn vị (`Position history`), bạn sẽ thiết kế cấu trúc dữ liệu nào và tích hợp nó vào đâu trong hệ thống hiện tại?
- Làm thế nào để thêm một phương thức `clone()` để sao chép một đơn vị quân sự (`Unit`) mà vẫn giữ được tính đa hình của các lớp con như `Vehicle` và `Infantry`?



5 Câu hỏi thiết kế mới (Mức nâng cao - Tính mở rộng và bảo trì) (CS)

1. Nếu hệ thống cần hỗ trợ thêm các loại đơn vị quân sự mới (ví dụ: máy bay hoặc tàu chiến), bạn sẽ thiết kế lại cấu trúc kế thừa hiện tại như thế nào để đảm bảo tính mở rộng và giảm thiểu việc sửa đổi mã cũ?
2. Làm thế nào để tích hợp một hệ thống sự kiện (event system) để thông báo khi quantity của một đơn vị thay đổi (ví dụ: do đào ngũ hoặc chi viện), và điều này có thể được áp dụng ra sao trong một hệ thống lớn hơn?
3. Nếu muốn tách logic tính toán `getAttackScore()` ra khỏi các lớp `Vehicle` và `Infantry` để dễ dàng thay đổi công thức tính điểm trong tương lai, bạn sẽ áp dụng mẫu thiết kế nào và triển khai cụ thể ra sao?
4. Trong trường hợp hệ thống cần hỗ trợ đa luồng (multi-threading) để xử lý nhiều đơn vị quân sự cùng lúc, bạn sẽ thiết kế lại các lớp này như thế nào để đảm bảo an toàn dữ liệu và hiệu suất?
5. Làm thế nào để thiết kế một hệ thống "module" cho phép thêm các thuộc tính động (dynamic attributes) vào các đơn vị quân sự mà không cần sửa đổi trực tiếp mã nguồn của lớp `Unit`, `Vehicle`, hay `Infantry`, đồng thời duy trì tính bảo trì và hiệu suất?

6 Design Patterns (mẫu thiết kế) và nguyên tắc SOLID (CS)

1. **Factory Pattern và Single Responsibility:** Làm thế nào để áp dụng mẫu **Factory Pattern** để tạo các đối tượng `Vehicle` và `Infantry` thay vì khởi tạo trực tiếp qua constructor, đồng thời đảm bảo lớp `Unit` chỉ chịu trách nhiệm cho các chức năng chung mà không phụ thuộc vào logic tạo đối tượng cụ thể?
2. **Strategy Pattern và Open/Closed:** Nếu công thức tính `getAttackScore()` cần thay đổi linh hoạt theo từng loại đơn vị hoặc ngữ cảnh chiến đấu, bạn sẽ sử dụng mẫu **Strategy Pattern** như thế nào để đảm bảo lớp `Vehicle` và `Infantry` tuân thủ nguyên tắc Open/Closed (mở để mở rộng, đóng để sửa đổi)?
3. **Decorator Pattern và Liskov Substitution:** Làm thế nào để áp dụng mẫu **Decorator Pattern** để thêm các khả năng đặc biệt (ví dụ: tăng sát thương, tăng phòng thủ) cho các đơn vị quân sự mà vẫn đảm bảo nguyên tắc Liskov Substitution (các lớp con như `Vehicle` và `Infantry` có thể thay thế hoàn toàn cho lớp cha `Unit`)?
4. **Dependency Inversion và Interface Segregation:** Nếu muốn tách logic hiển thị chuỗi của phương thức `str()` thành một giao diện riêng (interface), bạn sẽ thiết kế các giao diện và lớp phụ thuộc như thế nào để tuân thủ nguyên tắc Dependency Inversion và Interface Segregation, tránh việc các lớp phải triển khai những phương thức không cần thiết?
5. **Observer Pattern và Single Responsibility:** Làm thế nào để tích hợp mẫu **Observer Pattern** để thông báo cho các thành phần khác (ví dụ: giao diện người dùng hoặc hệ thống ghi log) khi trạng thái của một đơn vị (như quantity hoặc pos) thay đổi, đồng thời đảm bảo mỗi lớp chỉ chịu trách nhiệm cho một nhiệm vụ duy nhất theo nguyên tắc Single Responsibility?